

# TP2 — API — Météo changement d'API

## Objectif

Ce TP met à l'épreuve l'architecture construite au TP1. Si le couplage faible, l'IoC et la DI ont été respectés, chaque exercice se résout en ajoutant du code, presque sans en modifier. Le coût du changement est donc la mesure principale de ce TP.

## Changement d'API

Le client exige désormais un géocodeur souverain et souhaite pouvoir changer de fournisseur météo sans redéploiement.

### Services externes à intégrer

- **Géocodage** : API Adresse de la Base Adresse Nationale (adresse → latitude / longitude)
  - Exemple d'appel :  
`api-adresse.data.gouv.fr/search/?q=Alès&limit=1`
- **Météo** : MET Norway Locationforecast (latitude / longitude → prévisions)
  - Exemple d'appel :  
`api.met.no/weatherapi/locationforecast/2.0/compact?lat=44.12&lon=4.08`
  - **Attention : Un User-Agent identifiable est obligatoire** : Chaque requête doit porter un en-tête HTTP User-Agent qui identifie votre application et donne un moyen de vous contacter (adresse e-mail ou site web), par exemple « TP2-MeteoApi/1.0 prenom.nom@ecole.fr ». Une requête sans User-Agent, ou avec celui envoyé par défaut par votre bibliothèque HTTP (python-requests, okhttp, Java/...), est rejetée avec une erreur 403.

### Travail demandé

1. Implémentez un nouvel adaptateur de géocodage pour la BAN et un nouvel adaptateur météo pour MET Norway, derrière les abstractions existantes du TP1.
2. Rendez le choix du fournisseur configurable (variable d'environnement ou fichier de configuration), sans recompilation. Les deux anciens fournisseurs restent disponibles.

3. Écrivez une suite de tests de contrat unique, exécutée contre chaque implémentation d'une même abstraction (réponses simulées par bouchon HTTP) : adresse valide, adresse introuvable, réponse vide, caractères accentués.
4. Vérifiez que les objets propres à chaque API (DTO, noms de champs, formats) ne sortent pas de leur adaptateur.